

Unmanned driving integrated AI large model

1. Course Content

Note: This program runs on a sandbox map; you will need to adjust the program for your personal map.

This tutorial uses the factory-installed road sign and lane recognition models, primarily implementing autonomous driving road sign detection + map navigation + multimodal large model for intelligent control.

2. Course Prerequisites

- After completing the "Road Network Planning" chapter, save the road network navigation map and the road network map showing the car's trajectory.
- After completing the "AI Large Model Fundamentals" chapter, configure the large model's api_key and app_id.
- Obtain the pose data of the points the car needs to reach and store it in the large model's map_mapping.yaml file.
- Modify to road network navigation communication mode.

3. Configuration File Path

- Road Network Navigation Communication Mode

Open ~/.bashrc, uncomment this line, and change it as shown in the image. (After learning about the road network planning function, please restore this comment to avoid affecting the previous examples.)

```
vim ~/.bashrc
:wq #Save and exit
source ~/.bashrc #Update environment
```

```
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
#unset RMW_IMPLEMENTATION
export ROS_DOMAIN_ID=62
export ROBOT_TYPE=M1          # A1, M1
export RPLIDAR_TYPE=tmini     # c1, tmini
export CAMERA_TYPE=nuwa       # usb, nuwa

export INIT_SERVO_S1=90        # 0~180
export INIT_SERVO_S2=30        # 0~100

source /opt/ros/humble/setup.bash
source /home/sunrise/yahboomcar_ros2_ws/software/library_ws/install/setup.bash
source /home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/install/setup.bash

# Read the number of rows and columns from the terminal size
read -r rows cols <<(stty size)

:wq
```

- Road network navigation map and road network map

```
~/map
```

The image shows the saved map and road network diagram. The map.geojson file contains the road network diagram of the car's trajectory.

```
jetson@yahboom:~/map$ ls
map.data  map.geojson  map.pgm  map.posegraph  map.yaml  text
```

- Configure AI large model parameters

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/config/large_model_interface.yam
aml
```

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/config/yahboom.yaml
```

large_model_interface.yaml:

Select API according to the actual camera model

```
! large_model_interface.yaml
1 #大模型接口配置文件 Large model interface configuration file
2 #-----国内用户配置选项-----
3 #必须配置项
4 #通义千问平台API
5 #决策层大模型
6 tongyi_api_key : "api_key" #填写自己的api_key #决策层模型选用文本生成模型 配置方式参考 https://bailian.console.aliyun.com/?spm=5176.29597918.3_SEs5jsNv7
7 tongyi_app_id : "app_id" #填写自己的app_id
8 tongyi_base_url : "https://dashscope.aliyuncs.com/compatible-mode/v1" #通义接口地址, 无需修改
9 #执行层大模型
10 multimodel : "qwen-v1-max-2025-04-08" #模型选择多模态模型 模型列表参考 https://bailian.console.aliyun.com/?spm=5176.29597918.3_SEs5jsNv72yRuRfS
11 #测试较稳定模型: "qwen-v1-max", "qwen-v1-max-2025-04-08", "qwen-v1-max-2025-04-02"
12
13 #-----国外用户配置选项 / International version configuration options-----
14 #根据实际相机型号选择API / Select API according to the actual camera model
15 #USB camera parameters
16 #decision_AI_api_key : "app-1ztr3s3lEg470p0PLfnK0aRk" #dify决策大模型应用API_KEY
17 #execution_AI_api_key : "app-4ZA50UMQirPc3z3HCwY6s5De" #dify执行大模型应用API_KEY
18 # #uwa depth camera parameters
19 # #decision_AI_api_key : "app-1DGna1Unc4VhICrGh0awvch" #dify决策大模型应用API_KEY
20 # #execution_AI_api_key : "app-wi3fvhFFGjhtIrrvSgs6R0dx" #dify执行大模型应用API_KEY
21
22
23 #-----可选配置项 (非必须)-----
24 #online_asr配置
25 #online_asr_sample_rate: 16000 #asr音频采样率
26 #online_asr_model : "gummy-chat-v1" #部分语音识别模型只支持8k采样率 具体列表详见 https://bailian.console.aliyun.com/?spm=5176.29597918.3_SEs5jsNv72yRuRfS
27 # "paraformer-realtime-v2", "paraformer-realtime-v1", "paraformer-realtime-8k-v2", "paraformer-realtime-8k-v1", "gummy-realtime-v1", "gummy-chat-v1"
28
29
```

yahboom.yaml:

Select True for enable_route_nav so that the large model can invoke route navigation.

```
30 #-----国际版本参数 International version parameters-----
31 asr: #语音节点参数 # Voice node parameters
32   ros_parameters:
33     VAD_MODE: 2 #vad灵敏度 # VAD sensitivity
34     sample_rate: 16000 #asr录音音频采样率 # ASR recording audio sample rate
35     frame_duration_ms: 30 #vad帧大小单位ms # VAD frame size in milliseconds
36     use_online_asr: True #是否使用在线asr识别 # Whether to use online ASR recognition
37     mic_serial_port: "/dev/myspeech" #麦克风串口别名 # Microphone serial port alias
38     mic_index: 0 #麦克风索引 # Microphone index
39     language: 'en' #系统声音语言 # System sound language
40     regional_setting : "international" #international: 国际版 China: 国内版 # international: International version, China: Domestic version
41
42 action_service: #动作服务器节点参数 # Action server node parameters
43   ros_parameters:
44     Speed_topic: "/cmd_vel" #速度话题 # Speed topic
45     enable_route_nav: True #是否启动路网导航模式 # Whether to enable route navigation mode
46     text_chat_mode: false #文字交互模式 # Text chat mode
47     use_onlinetts: True #是否使用在线语音合成 # Whether to use online text-to-speech synthesis
48     language: 'en' #本地语音合成语言 # local text-to-speech synthesis language
49     regional_setting : "international" #international: 国际版 China: 国内版 # international: International version, China: Domestic version
50
51 model_service: #模型服务器节点参数 # Model server node parameters
52   ros_parameters:
53     language: 'en' #大模型接口语言 # Large model API language
54     regional_setting : "international" #international: 国际版 China: 国内版 # international: International version, China: Domestic version
55
```

- Configure the pose data of the target points

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/config/map_mapping.yaml
```

Add new target points under the route navigation section, as this course uses route navigation.

```
#此处可新增添加区域对应的栅格地图坐标点，注意和上面格式保持一致
#Here, you can add the raster map coordinate points corresponding to the added area. Please note that the format should be consistent with the above
road_net_map_areas: #路网导航 road network navigation
A:
  name: '五金店'
  position:
    x: 1.1409525871276855
    y: -0.6988544464111328
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: 0.7420813339272786
    w: 0.6703098491270368
B:
  name: '数字楼'
  position:
    x: 2.229278564453125
    y: -0.5782679915428162
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -0.999210671704489
    w: 0.03972447044157906
C:
  name: '学校'
  position:
    x: 1.520
    y: 0.124
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: 0.093
    w: 0.996
D:
  name: '工地'
  position:
    x: 0.006
    y: -1.546
    z: 0.0
  orientation:
    x: 0.0
```

If you don't know how to obtain the pose data of a point, you can first refer to the "04-Route Planning Combined with Sandbox Map Case" in the route planning navigation section.

After modifying the yaml files, you need to recompile and then run the program.

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws
colcon build --packages-select largemodel
```

4. Program Startup

4.1 Startup Command

- Launch large model + chassis + camera

```
ros2 launch largemodel largemodel_control.launch.py
```

```
jetson@yahboom: ~/map
jetson@yahboom: ~/map 80x23
[ascamera_node-1] [INFO] [1764642930.680828572] [ascamera_hp60c.camera_publisher]: 2025-12-02 10:35:30[INFO] [XuCmdCameraHp60c.cpp] [458] [getMjpegSize] mjpeg size:640x480
[ascamera_node-1] [INFO] [1764642930.680961942] [ascamera_hp60c.camera_publisher]: camera opened
[ascamera_node-1] [INFO] [1764642930.681406530] [ascamera_hp60c.camera_publisher]: get config info, ret 0, is_Registration 1
[ascamera_node-1] [INFO] [1764642930.683420777] [ascamera_hp60c.camera_publisher]: #camera[0xaaaabda23ec0] SN[ASC60CF19001984]'s firmware version:DLS_VERc20865=:V01.09_2024110714:V01.19_2024110714:L01
[ascamera_node-1] [INFO] [1764642930.684469979] [ascamera_hp60c.camera_publisher]: set depth resolution: 640 x 480 @ 25fps
[ascamera_node-1] [INFO] [1764642930.685288791] [ascamera_hp60c.camera_publisher]: set rgb resolution: 640 x 480 @ 25fps
[ascamera_node-1] [INFO] [1764642930.695994045] [ascamera_hp60c.camera_publisher]: 2025-12-02 10:35:30[INFO] [CameraHp60c.cpp] [259] [startStreaming] start streaming
[ascamera_node-1] [INFO] [1764642930.696164725] [ascamera_hp60c.camera_publisher]: 2025-12-02 10:35:30[INFO] [CameraSrv.cpp] [206] [onAttached] attached end
[ascamera_node-1] [INFO] [1764642931.701658854] [ascamera_hp60c.camera_publisher]: 2025-12-02 10:35:31[INFO] [CameraHp60c.cpp] [278] [stopStreaming] stop streaming
```

V

- Enter the road network navigation docker

```
bringup_ros2kilted
```

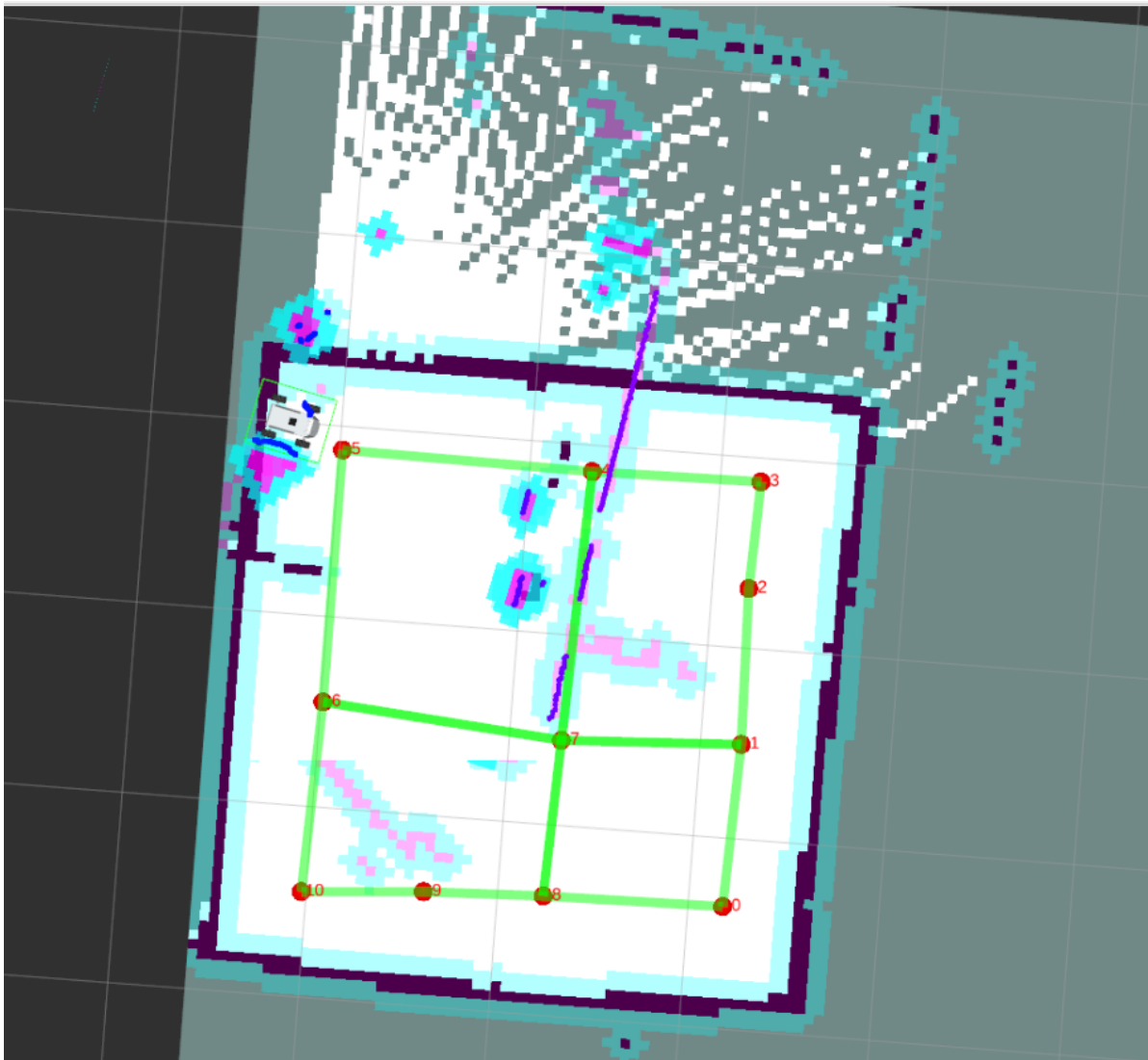
```
jetson@yahboom:~$ bringup_ros2kilted
Access control disabled, clients can connect from any host
+] Running 1/1
✓ Container ros2_kilted-ros2_kilted-1 Started 1.8s
jetson@yahboom:~$
```

```
exec_ros2kilted
```

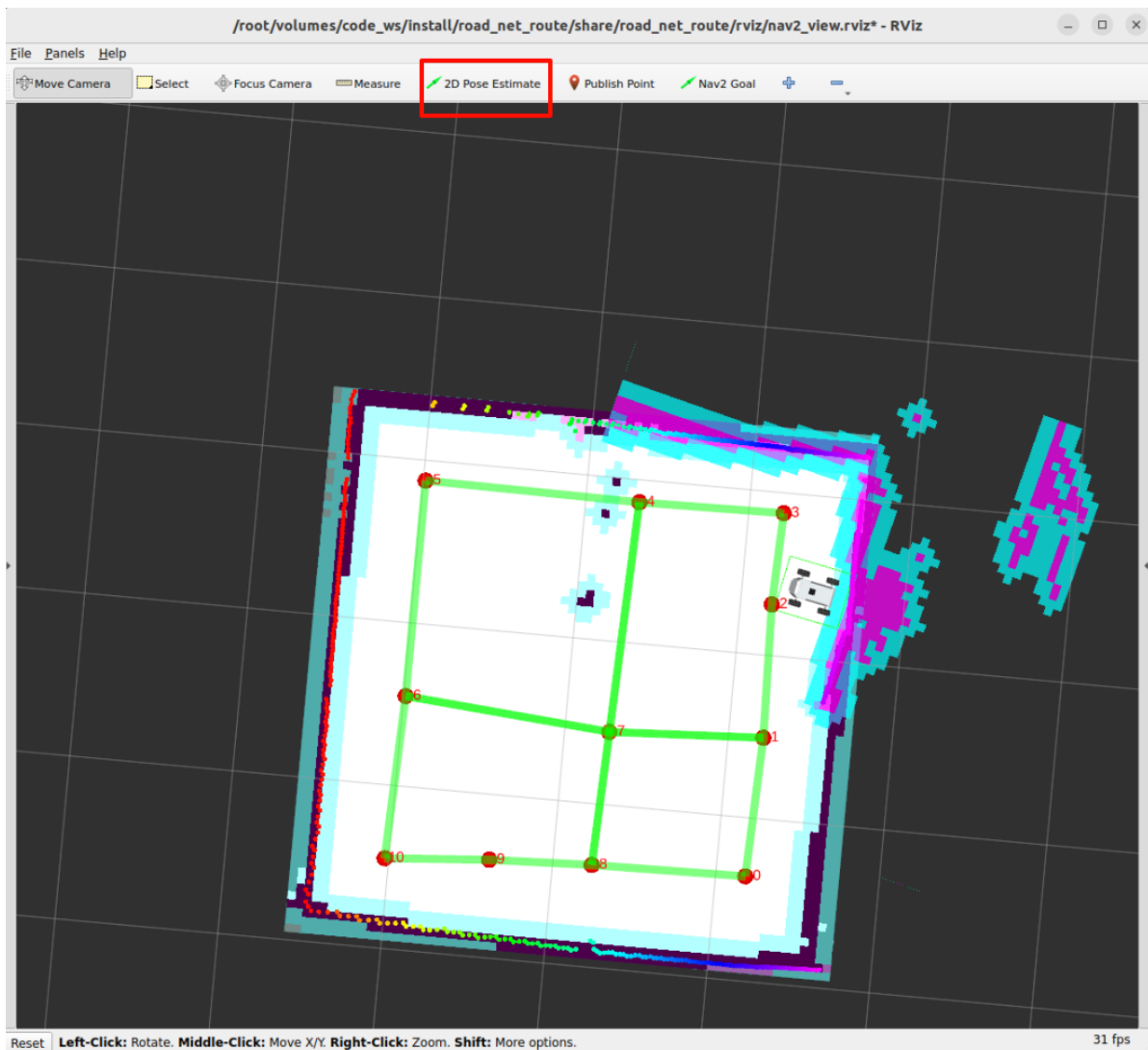
```
jetson@yahboom:~$ exec_ros2kilted
root@yahboom:/#
```

Enter this command in the terminal.

```
ros2 launch road_net_route roadnet_nav2.launch.py
```



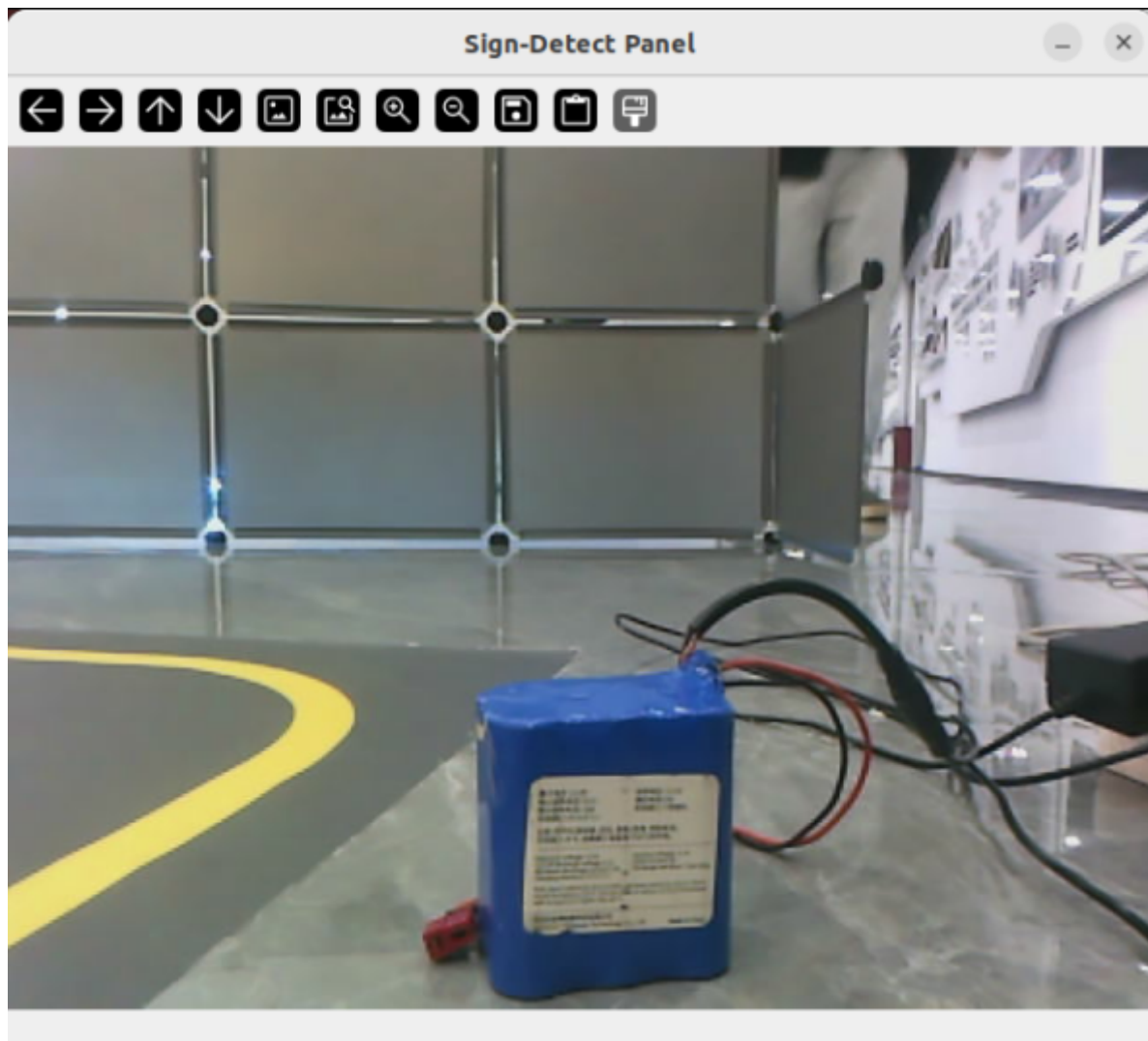
Upon entering, you'll notice that the point cloud and the map don't overlap; you'll need to use 2D Pose Estimate to reposition the car.



- Start YOLO Road Sign Recognition

```
ros2 launch auto_drive auto_drive.launch.py
```

After successful launch, a visual interface will appear.



4.2 Test Cases

Here are some reference test cases. Users can create their own dialogue commands.

- Navigate to the construction site while following traffic rules

4.2.1 Case 1

First, wake up the robot with "Hello yahboom". The robot will respond with "I'm here, please give instructions". After the robot responds, the buzzer will sound briefly (beep—), then you can speak. The robot will perform voice activity detection (VAD); if there is voice activity, it will print 1, otherwise it will print -. When speaking ends, it will perform tail silence detection; if silent for more than 450ms, it will stop recording.

Dynamic voice activity detection (VAD) is shown in the figure below:

```
jetson@yahboom: ~/yahboomcar_ros2_ws/yahboomcar_ws
jetson@yahboom: ~/yahboomcar_ros2_ws/yahboomcar_ws 92x30
[asr-16] [INFO] [1764657278.481868423] [asr]: -
[asr-16] [INFO] [1764657278.542368095] [asr]: 1
[asr-16] [INFO] [1764657278.602056950] [asr]: 1
[asr-16] [INFO] [1764657278.663014816] [asr]: 1
[asr-16] [INFO] [1764657278.723742529] [asr]: 1
[asr-16] [INFO] [1764657278.783522940] [asr]: 1
[asr-16] [INFO] [1764657278.844035892] [asr]: 1
[asr-16] [INFO] [1764657278.904690834] [asr]: 1
[asr-16] [INFO] [1764657278.965337647] [asr]: -
[asr-16] [INFO] [1764657279.026852175] [asr]: 1
[asr-16] [INFO] [1764657279.086691627] [asr]: 1
[asr-16] [INFO] [1764657279.148074021] [asr]: 1
[asr-16] [INFO] [1764657279.208527417] [asr]: -
[asr-16] [INFO] [1764657279.268960685] [asr]: 1
[asr-16] [INFO] [1764657279.329160439] [asr]: -
[asr-16] [INFO] [1764657279.389803891] [asr]: -
[asr-16] [INFO] [1764657279.450415982] [asr]: 1
[asr-16] [INFO] [1764657279.511227697] [asr]: 1
[asr-16] [INFO] [1764657279.571453372] [asr]: 1
[asr-16] [INFO] [1764657279.632190108] [asr]: 1
[asr-16] [INFO] [1764657279.692880410] [asr]: 1
[asr-16] [INFO] [1764657279.753505045] [asr]: 1
[asr-16] [INFO] [1764657279.814180307] [asr]: 1
[asr-16] [INFO] [1764657279.874934707] [asr]: -
[asr-16] [INFO] [1764657279.934630313] [asr]: -
[asr-16] [INFO] [1764657279.995254692] [asr]: 1
[asr-16] [INFO] [1764657280.055911840] [asr]: 1
[asr-16] [INFO] [1764657280.124965099] [asr]: -
[asr-16] [INFO] [1764657280.185932819] [asr]: -
[asr-16] [INFO] [1764657280.246335620] [asr]: -
```

The robot first recognizes the user's speech, then makes decisions and takes actions according to the instructions. Simultaneously, the terminal prints the following information:

```
jetson@yahboom: ~/yahboomcar_ros2_ws/yahboomcar_ws
jetson@yahboom: ~/yahboomcar_ros2_ws/yahboomcar_ws 92x30
[asr-16] [INFO] [1764657605.668674285] [asr]: 1
[asr-16] [INFO] [1764657605.729629067] [asr]: 1
[asr-16] [INFO] [1764657605.790110264] [asr]: 1
[asr-16] [INFO] [1764657605.849843115] [asr]: -
[asr-16] [INFO] [1764657605.911233176] [asr]: -
[asr-16] [INFO] [1764657605.972604740] [asr]: -
[asr-16] [INFO] [1764657606.033191989] [asr]: -
[asr-16] [INFO] [1764657606.093282964] [asr]: -
[asr-16] [INFO] [1764657606.153825827] [asr]: -
[asr-16] [INFO] [1764657606.213770717] [asr]: -
[asr-16] [INFO] [1764657606.274789948] [asr]: -
[asr-16] [INFO] [1764657606.335223463] [asr]: -
[asr-16] [INFO] [1764657606.395954556] [asr]: -
[asr-16] [INFO] [1764657606.458824763] [asr]: -
[asr-16] [INFO] [1764657606.518130303] [asr]: -
[asr-16] [INFO] [1764657606.577540327] [asr]: -
[asr-16] [INFO] [1764657606.640408294] [asr]: -
[asr-16] [INFO] [1764657606.698531713] [asr]: -
[asr-16] [INFO] [1764657606.757904103] [asr]: -
[asr-16] [INFO] [1764657606.818232686] [asr]: -
[asr-16] [INFO] [1764657606.878078981] [asr]: -
[asr-16] [INFO] [1764657606.939070019] [asr]: -
[asr-16] [INFO] [1764657606.998873624] [asr]: -
[asr-16] [INFO] [1764657607.060230339] [asr]: -
[asr-16] [INFO] [1764657607.120183356] [asr]: -
[asr-16] [INFO] [1764657608.765174546] [asr]: -
[asr-16] [INFO] [1764657608.765862122] [asr]: 😊okay, let me think for a moment...
[model_service-14] [INFO] [1764657610.791974513] [model_service]: 决策层AI规划:1. 导航去工地
[model_service-14] 2. 途中遵守交通规则
```

The decision-making layer's large model outputs the planned task steps:

```
[model_service-14] [INFO] [1764657284.944787510] [model_service]: 决策层AI规划:1. 导航去工地
[model_service-14] 2. 途中遵守交通规则
```

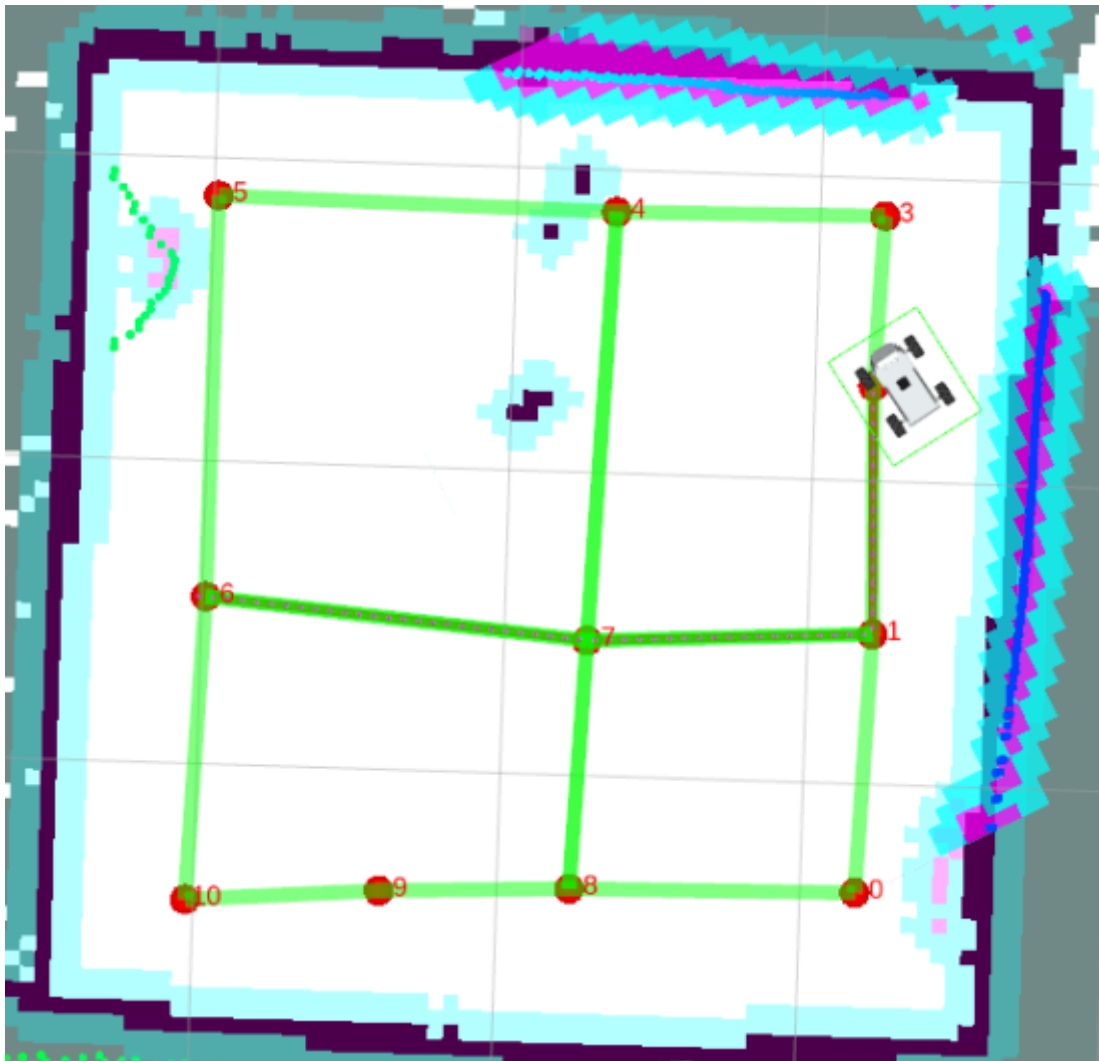
Then, the execution layer's large model executes according to these task steps:

```

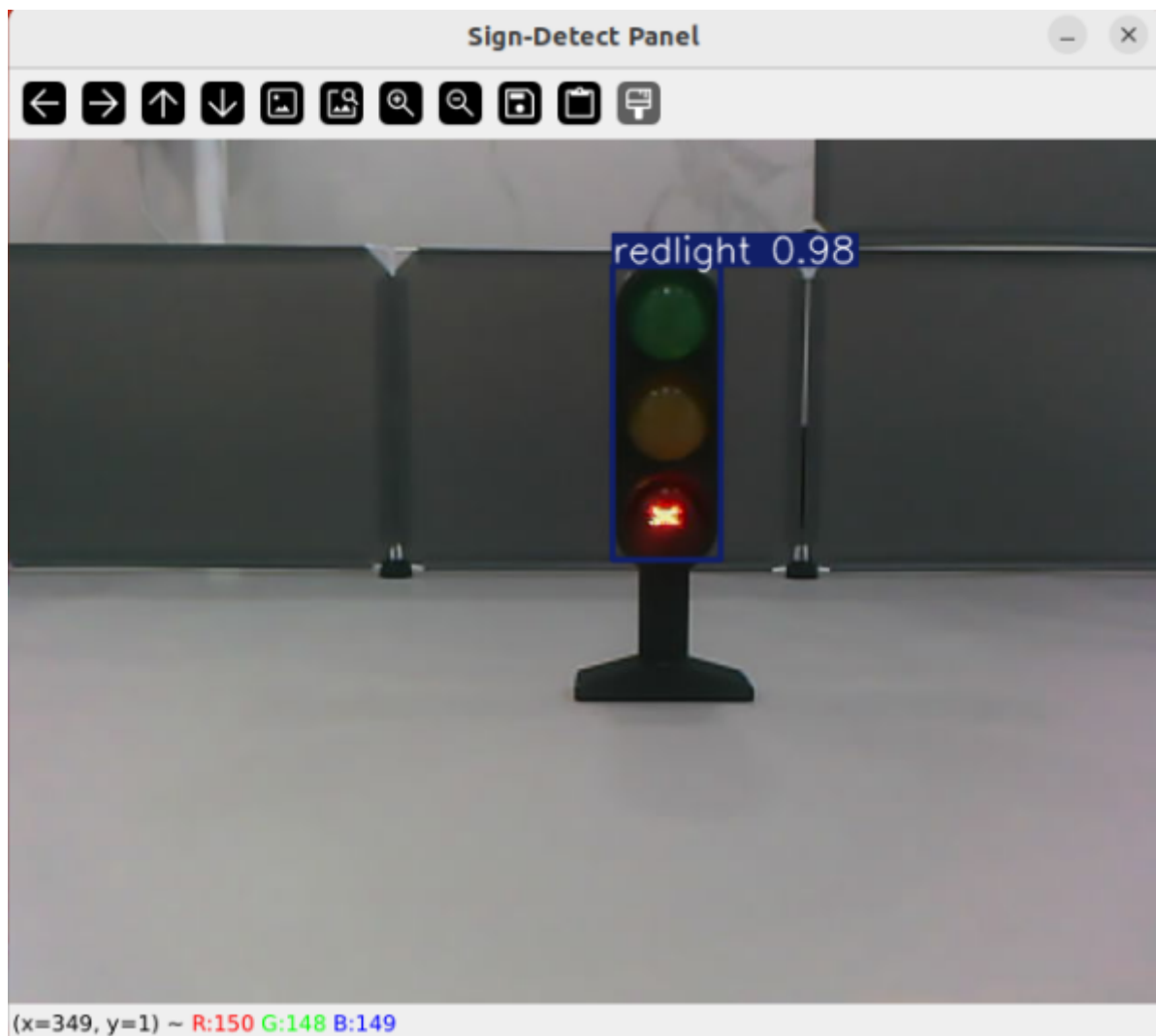
[action_service_nuwa-15] [INFO] [1764658284.847215058] [action_service]: Published message:
机器人反馈:执行control_sign(enable)完成
[action_service_nuwa-15] [INFO] [1764658284.858613098] [action_service]: Published message:
机器人反馈:执行['navigation(D)', 'control_sign(enable)']完成
[model_service-14] [INFO] [1764658286.053808441] [model_service]: "action": [], "response":
交通规则已开启, 现在安全出发去工地
[model_service-14] [INFO] [1764658287.629069342] [model_service]: "action": ['finishtask()']
, "response": 已到达工地, 任务完成!
[action_service_nuwa-15] [INFO] [1764658291.935198082] [action_service]: Published message:
机器人反馈:回复用户完成
[model_service-14] [INFO] [1764658293.260168282] [model_service]: "action": ['finishtask()']
, "response": 已到达工地, 任务完成!

```

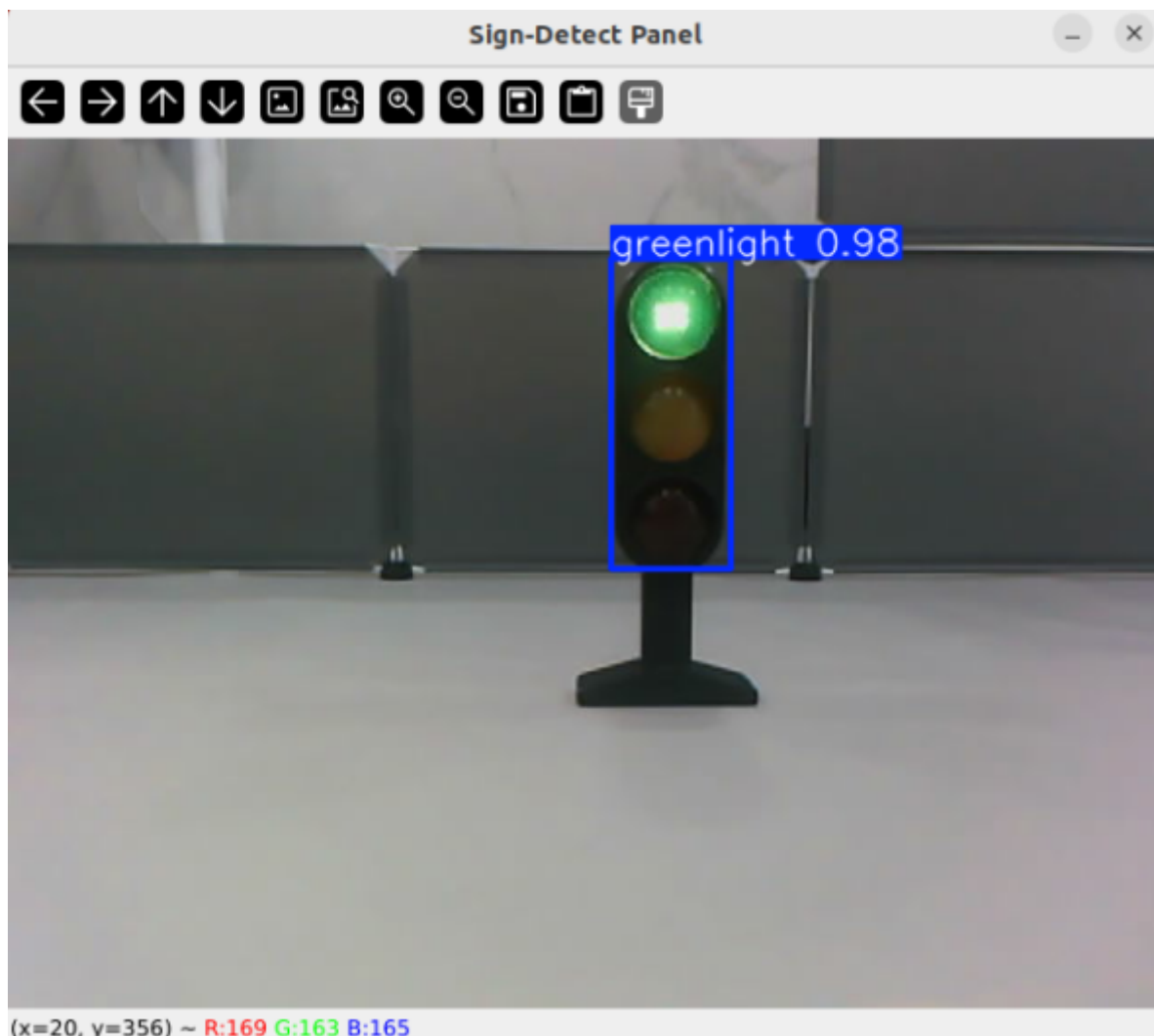
Simultaneously, as you can see, the road network navigation interface displays a grid route for the car to navigate to the construction site.



If we turn on a red light midway through the navigation, the car will stop and wait for the green light before resuming navigation.



When the green light comes on, the car will continue navigating. **Note that the traffic lights should not be placed too close to the car's route; if they are too close, they may be perceived as obstacles and affect navigation.**



5. Source Code Analysis

5.1 Path to the Large Model Source Code

```
# jetson orin nano users
~/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_service_nuwa
.py
# RDK X5 users
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_
service_nuwa.py
```

action_service_nuwa.py: This file contains the main logic for the large model integration.

Key functionalities include:

- Providing action servers to handle navigation and robot control commands
- Integrating speech recognition and speech synthesis
- Enables switching between road network navigation and regular navigation
- Handles large language model interactions

It supports two navigation modes: **Regular Navigation:** a standard navigation mode based on a single point target; and **Road Network Navigation:** a navigation mode based on a predefined road network path. To integrate with autonomous driving, `enable_route_nav: True` needs to be enabled for road network navigation.

Navigation Process:

- Receive navigation request
- Select navigation strategy based on current mode (Road Network/Regular)
- Execute navigation actions
- Provide feedback and results

```
# In the init_param_config method
self.declare_parameter("enable_route_nav", False)
self.enable_route_nav =
self.get_parameter("enable_route_nav").get_parameter_value().bool_value
```

Navigation mode switching:

```
def handle_navigation_request(self, destination):
    """Handling navigation requests"""
    if self.enable_route_nav:
        # Road network navigation mode
        if destination in self.route_network:
            self.follow_route(self.route_network[destination])
        else:
            self.get_logger().warn(f"No route found for destination:
{destination}")
    else:
        # Normal navigation mode
        self.navigate_to_waypoint(destination)
```

5.2 Road Network Navigation Source Code Path

```
#jetson orin nano
~/ros2_kilted/volumes/code_ws/src/road_net_route/launch/roadnet_nav2.launch.py
#RDK X5
/home/sunrise/ros2_kilted/volumes/code_ws/src/road_net_route/launch/roadnet_nav2
.launch.py
```

roadnet_nav2.launch.py:

The launch file will directly call the file saved in the /root/map path. If it has a different name, you can change the name here and then recompile and run it.

```
declare_map_yaml_cmd = DeclareLaunchArgument(
    'map',
    default_value='/root/map/map.yaml',
)
declare_pose_map_cmd = DeclareLaunchArgument(
    'pose_map',
    default_value='/root/map/map',
)
declare_roadnet_file_cmd = DeclareLaunchArgument(
    'roadnet_file',
    default_value='/root/map/map.geojson',
    description=".geojson格式路网文件路径",
)
```

The parameter file used for navigation.

```
declare_params_file_cmd = DeclareLaunchArgument(
    'params_file',
    default_value=os.path.join(get_package_share_directory('road_net_route'),
    'config', 'nav2_kilted.yaml'),
    description='Full path to the ROS2 parameters file to use for all
    launched nodes',
    )
```

You can open the nav2_kilted.yaml file in this path and modify some navigation parameters. Normally, you don't need to modify the parameters; most of them are pre-tuned and don't require modification.

```
#jetson orin nano
~/ros2_kilted/volumes/code_ws/src/road_net_route/config/nav2_kilted.yaml
#RDK X5
~/ros2_kilted/volumes/code_ws/src/road_net_route/config/nav2_kilted.yaml
```

5.3 YOLO recognition source code

```
#jetson orin nano
~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/launch/auto_drive.launch.py
#RDK X5
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/launch/auto_drive.
launch.py
```

auto_drive.launch.py:

The YOLO system is used to recognize road signs and instruct the car to perform corresponding actions. However, because this is based on a road network navigation system, the right turn and stop road signs will be disabled, as these two signs will interfere with the road network navigation.

Note that road network navigation needs to be enabled in the YAML file.

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/config/auto_drive_node.yaml
```

```
auto_drive:
  ros__parameters:
    turn_radius: 0.4
    drive_speed: 0.15
    angular_speed: -0.72
    duration: 1.3
    response_distance_1: 2900
    response_distance_2: 132
    cooldown_time: 3.0
    DEBUG_MODE: False
    START_AutoDrive: True
    combine_nav: True
    Kp: -0.62
    Ki: 0.0
    Kd: -0.1
    max_integral: 1000.0
    image_size: [640, 480]
```

